

Design and HLS Implementation of a Custom CNN Accelerator

Joon Hyuk Song

Charles W. Davidson College of Engineering, San Jose State University

joonhyuk.song@sjsu.edu

Abstract— Field-Programmable Gate Arrays (FPGAs) offer high energy efficiency and hardware parallelism for edge deep learning acceleration. This paper demonstrates an end-to-end design methodology for a quantized Convolutional Neural Network (CNN) performing MNIST digit classification on a Xilinx Zynq-7000 Zybo platform. The model is trained and quantized in Python, exported, and synthesized into hardware IP using AMD Vitis High-Level Synthesis (HLS). Integrated via AXI bus protocols, post-implementation timing analysis confirms the design operates at a target clock frequency of 100 MHz, achieving a critical path delay of 9.278 ns with a positive timing slack of 0.722 ns. Although deeply nested conditionals in the argmax layer dictated the critical path, the architecture meets all timing constraints while highlighting trade-offs in DSP resource allocation.

Keywords— High-Level Synthesis (HLS), Convolutional Neural Network (CNN), FPGA, AXI protocols

I. INTRODUCTION

Hardware acceleration of Deep Neural Networks on Field-Programmable Gate Arrays (FPGAs) has emerged as a primary research focus in modern embedded systems and computer architecture. This surge in interest is largely driven by the exceptional energy efficiency and low latency of FPGAs, highly suitable for power-constrained edge devices. Beyond their dedicated edge inference engines, modern FPGAs play vital roles in ASIC design verification and hardware prototyping, reducing cost and mitigating the risk of post-silicon failure.

The primary objective of this project was to design, synthesize, and verify a custom Convolutional Neural Network (CNN) accelerator IP core optimized for MNIST handwritten digit classification (28×28 grayscale images, digits 0 to 9). The software-to-hardware co-design framework leverages TensorFlow for Quantization-Aware Training (QAT) and fixed-point parameter export, while High-Level Synthesis (HLS) was employed to implement, optimize, and functionally verify the hardware IP core.

Step 1: Software Modeling & Quantization

A Convolutional Neural Network (CNN) model was trained on the MNIST dataset using

Quantization-Aware Training (QAT) in TensorFlow, followed by offline Batch Normalization folding into the convolutional weight and bias parameters.

Step 2: High-Level Synthesis (HLS) Hardware Design

C/C++ hardware architectures were developed in Vitis HLS, comprising dedicated Convolutional Neural Network (CNN) and Fully Connected Neural Network (FNN) Modules integrated into a unified top function.

Step 3: Verification & Co-Simulation

The hardware design was validated using a custom HLS testbench and a sample input image. Functional accuracy was confirmed via C simulation, followed by C/RTL co-simulation to verify resource utilization and timing.

Step 4: IP Core Packaging & Export

Upon successful synthesis and verification, the design was packaged and exported as a reusable AMD Vivado IP core, ready for Vivado block design.

II. NETWORK ARCHITECTURE & TRAINING

A. Data Preprocess

The MNIST dataset obtained from the TensorFlow library—comprising 50,000 training, 10,000 validation, and 10,000 test images of handwritten digits (0–9)—was preprocessed in row-major order. The 8-bit unsigned integer pixel intensities [0, 255] were concurrently normalized to a [0, 1] floating-point range by dividing each value by 255.

B. Model Summary

The network topology consists of a 2-layer, 2-channel Convolutional Neural Network (CNN) followed by a Feedforward Neural Network (FNN) classifier. Each convolutional layer uses 3×3 kernels followed by 2×2 max-pooling. The classifier flattens the feature map into a dense layer of 98 neurons, fully connected to 10 output class neurons, where an argmax function selects the predicted digit. For FPGA deployment, the model was trained using Quantization-Aware Training (QAT) with 8-bit signed weights and 16-bit signed biases. Batch Normalization (BN) parameters were

folded directly into the weight and bias tensors post-training. The complete model pipeline is illustrated in Figure 1, and the detailed bit-widths and parameter counts are summarized in Table I

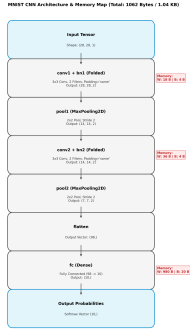


Figure 1

Table I

Layer	Weight (8bit)	Bias (16bit)
Conv1	3x3 kernel 2 channels =18 bytes	2 channels =4 bytes
Conv2	3x3x2 kernel 2 channels =36 bytes	2 channels =4 bytes
FNN	98 neurons, 10 outputs = 980 bytes	10 neurons =20 bytes

C. Results

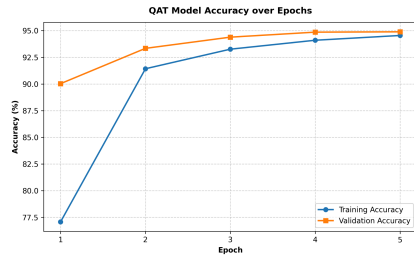


Figure 2

MNIST Confusion Matrix (Test Accuracy: 94.94%)

	0	1	2	3	4	5	6	7	8	9
0	947	0	2	2	0	9	10	4	6	0
1	0	1107	6	3	1	1	4	0	13	0
2	3	3	961	21	3	3	0	6	30	2
3	0	0	2	980	1	10	0	5	11	1
4	2	2	4	0	942	0	4	3	4	21
5	1	0	0	16	0	859	5	1	10	0
6	10	3	4	0	8	14	909	0	10	0
7	3	2	23	16	1	1	0	954	8	20
8	6	1	11	23	3	15	1	6	901	7
9	9	6	1	10	11	18	0	12	8	934
	0	1	2	3	4	5	6	7	8	9

Figure 3

III. HIGH-LEVEL SYNTHESIS

A. Archeticture Overview & Finite State Machine(FSM)

The proposed mnist_ip core is implemented in Vitis HLS using the C++ source file and top-level header. Fig. 4 illustrates the system-level hardware architecture of the accelerator IP core. Input bitmap images, along with network weights and biases, are stored in off-chip DDR system memory. During inference execution, input images are transferred from DDR via an AXI Direct Memory Access (DMA) engine using an AXI4-Master interface and streamed directly into the IP core through an AXI4-Stream interface.

Model weights and biases are fetched into the core's internal static Block RAM via a dedicated AXI4-Master Memory-Mapped interface. For intermediate layer feature maps, an external scratchpad BRAM connects directly to the IP core through a point-to-point BRAM interface with dual read and write capability. Within Vivado Block Design, these memory channels are decoupled via AXI Interconnect.

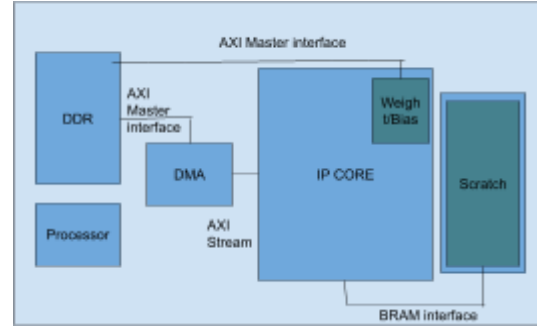


Figure 4

To control the accelerator execution flow within a single engine, an 8-state Finite State Machine (FSM) is implemented. The states include IDLE, LOAD_W, CONV1, WAIT_CONV2, CONV2, WAIT_FC, FC, and DONE. Table II outlines each state's functional responsibility in the pipeline.

TABLE II

Finite State Machine (FSM) State Descriptions

State	Functional Role

IDLE	Initial default state
LOAD_W	Fetches weight and bias parameters from DDR memory into internal static BRAM (weight_bram).
CONV1	Reads the input image stream and computes the first convolutional layer features.
WAIT_CON V2	Holds execution until the host updates the control register to trigger the second layer.
CONV2	Reads intermediate feature maps from scratch BRAM and executes the second convolutional layer.
WAIT_FC	Holds execution until the host updates the control register to trigger the fully connected layer.
FC	Reads processed features from scratch BRAM and executes the dense layer for final prediction.
DONE	Asserts completion flags and prepares the core to transition back for subsequent tasks.

Because the intermediate feature map contains two pixel channels requiring 16 bits of total storage, and the external Block RAM (BRAM) primitive in Vivado is configured for a 32-bit data width, the scratch BRAM words are formatted to 32 bits with zero-padding. Appendix Fig. A.1 illustrates the top-level hardware interfaces, while Appendix Fig. A.2 details the memory-mapped register offsets.

IV. VERIFICATION

The proposed mnist_ip core was verified in Vitis HLS and Vivado XSim using a C/C++ testbench executing on a sample digit '5' bitmap (.bmp) with model parameters in simulated DDR memory.

Interrupt-Driven Flow: Weight loading (LOAD_W) executes initially. Upon detecting an interrupt, the host testbench polls status_reg (0x28) and writes the appropriate layer and start to ctrl_reg (0x20) to unblock and trigger the next state.

Layer Execution: The 784-pixel image stream is processed via AXI4-Stream during CONV1. Intermediate feature maps are written to and read from external dual-port scratch BRAM across CONV1, CONV2, and FC execution.

Co-Simulation Results: For the digit '5' test image, the hardware correctly classified the sample, asserting prediction_ap_vld with a final value of 0x5 in the prediction register.

V. RESULTS & RESOURCE UTILIZATION

Following post-implementation timing analysis, the design achieved a critical path delay of 9.278 ns, satisfying the 10.0 ns required clock period for the 100 MHz target frequency with positive timing slack (0.722 ns). The critical path was located within the final layer's argmax module, driven by deeply nested conditional if-statements. As detailed in Table B.1, resource utilization revealed that DSP block usage exceeded the recommended allocation threshold.

VI. Conclusion

This work successfully demonstrated an end-to-end hardware-software co-design framework for a quantized Convolutional Neural Network (CNN) targeted at MNIST digit classification on the Xilinx Zynq-7000 Zybo FPGA platform. By leveraging Quantization-Aware Training (QAT) in TensorFlow with 8-bit weights and 16-bit biases, alongside offline Batch Normalization folding, the network model was optimized for low-bitwidth fixed-point hardware execution.

The custom mnist_ip accelerator core, developed using AMD Vitis High-Level Synthesis (HLS) and coordinated by an 8-state Finite State Machine (FSM), seamlessly integrated AXI4-Stream, AXI4-Master, AXI4-Lite, and external dual-port scratchpad BRAM interfaces. Post-implementation results confirmed that the system operates reliably at its 100 MHz target frequency, achieving a critical path delay of 9.278 ns with a positive timing slack of 0.722 ns.

While the design fully satisfied timing constraints and validated functional accuracy during C/RTL co-simulation, the analysis highlighted key area and timing trade-offs:

Timing Bottleneck: The critical path was dictated by deeply nested if conditionals inside the argmax classification module.

Resource Allocation: DSP resource usage reached 90.00%, and LUT Combining reached 22.69%, exceeding standard optimization guidelines.

VII. Appendix.A

TABLE A.1
Top-Level IP Interface Summary

Interface Port	Protocol / Type	Direction	Data Width	Address Width	Description
m_axi_gmem	AXI4-Master	Read-Only	32-bit	64-bit	Burst weight/bias data fetch from DDR memory
s_axi_control	AXI4-Lite	Slave	32-bit	5-bit	Base offset registers for memory pointers
s_axi_ctrl	AXI4-Lite	Slave	32-bit	7-bit	Core control, status, and prediction registers
input_stream	AXI4-Stream	Input	8-bit	N/A	High-throughput streaming pixel data input
scratch_PORTA	BRAM Interface	In/out	32-bit	32-bit	Dual-port read/write access to external scratch BRAM

TABLE A.2
Memory-Mapped Control Register Addresses

Interface	Offset Address	Register Name	Access	Width	Bit Fields & Functional Description
s_axi_ctrl	0x00	CTRL	R/W	32-bit	Bit 0: ap_start Bit 1: ap_done Bit 2: ap_idle Bit 3: ap_ready Bit 7: auto_restart Bit 9: interrupt
s_axi_ctrl	0x04	GIER	R/W	32-bit	Bit 0: Global Interrupt Enable (1 = Enable)
s_axi_ctrl	0x08	IP_IER	R/W	32-bit	Bit 0: Channel 0 Interrupt Enable (CHAN0_INT_EN)
s_axi_ctrl	0x0C	IP_ISR	R/W	32-bit	Bit 0: Channel 0 Interrupt Status (CHAN0_INT_ST)
s_axi_ctrl	0x10	prediction	R	32-bit	Lower 4 bits contain output digit prediction (0–9)
s_axi_ctrl	0x14	prediction_ctrl	R	32-bit	Bit 0: Data Valid flag (prediction_ap_vld)
s_axi_ctrl	0x20	ctrl_reg	W	32-bit	FSM State Control register (written by CPU host)
s_axi_ctrl	0x28	status_reg	R	32-bit	Accelerator status feedback word

s_axi_ctrl	0x2C	status_reg_ctrl	R	32-bit	Bit 0: Data Valid flag (status_reg_ap_vld)
s_axi_ctrl	0x38	intr	R	32-bit	Dedicated interrupt status boolean signal
s_axi_ctrl	0x3C	intr_ctrl	R	32-bit	Bit 0: Data Valid flag (intr_ap_vld)
s_axi_ctrl	0x48	weight_addr	W	32-bit	DDR memory offset address for weights/biases
s_axi_control	0x10	ddr_1	W	32-bit	DDR base physical address (Bits [31:0])
s_axi_control	0x14	ddr_2	W		

VIII. Appendix.B

Table B.1
Post-Implementation Resource Utilization

Criteria	Guideline	Actual	Status
LUT	70%	59.68%	OK
FD	50%	20.57%	OK
LUTRAM+SRL	25%	7.58%	OK
MUXF7	15%	13.76%	OK

LUT Combining	20%	22.69%	REVIEW
DSP	80%	90.00%	REVIEW
RAMB/FIFO	80%	10.83%	OK
DSP+RAMB+URAM (Avg)	70%	50.41%	OK
BUFGCE* + BUFGCTRL	24	0	OK
DONT_TOUCH (cells/nets)	0	0	OK
MARK_DEBUG (nets)	0	0	OK
Control Sets	330	331	REVIEW
Average Fanout for modules > 100k cells	4	2.99	OK
Max Average Fanout for modules > 100k cells	4	0	OK
Non-FD high fanout nets > 10k loads	0	0	OK

IX. REFERENCE